

CSCI 650 - Assignment 4

Sean Angrisani | October 6, 2026

Preparation note: implementation, experiments, and this solution draft were prepared with AI assistance. The assignment calls for independently written solutions; review the explanations and follow the course's disclosure rules before submitting.

1(a). Total tour distance

The route lists each city once. Sum the directed distances between consecutive cities and include the final edge back to the first city. Empty routes have distance zero. The calculation takes $O(n)$ time and $O(1)$ auxiliary space.

```
def totalDistance(D, path):
    """Return the closed-tour distance, including the last-to-first edge."""
    if not path:
        return 0.0
    return float(sum(D[path[j]][path[(j + 1) % len(path)]]
                     for j in range(len(path))))
```

1(b). Nearest-neighbor heuristic

Start at the requested city, or choose a random city when start is -1. Repeatedly select the closest unvisited city; ties go to the smaller city index. The returned route contains each city exactly once.

```
def nearestNeighbor(D, start=-1):
    """Visit each city once, greedily choosing the closest unvisited city."""
    n = len(D)
    if n == 0:
        return []
    if start == -1:
        start = int(np.random.randint(n))
    if not 0 <= start < n:
        raise ValueError("start must identify a city")
    path = [start]
    remaining = set(range(n))
    remaining.remove(start)
    while remaining:
        city = min(remaining, key=lambda v: (D[path[-1]][v], v))
        path.append(city)
        remaining.remove(city)
    return path
```

At successive steps the algorithm examines $n-1, n-2, \dots, 1$ remaining cities. Their sum is $n(n-1)/2$, so the runtime is $\Theta(n^2)$, assuming constant-time matrix access and expected constant-time set operations. Auxiliary space is $O(n)$. This greedy rule does not guarantee an optimal tour.

1(c). Nearest-neighbor experiments

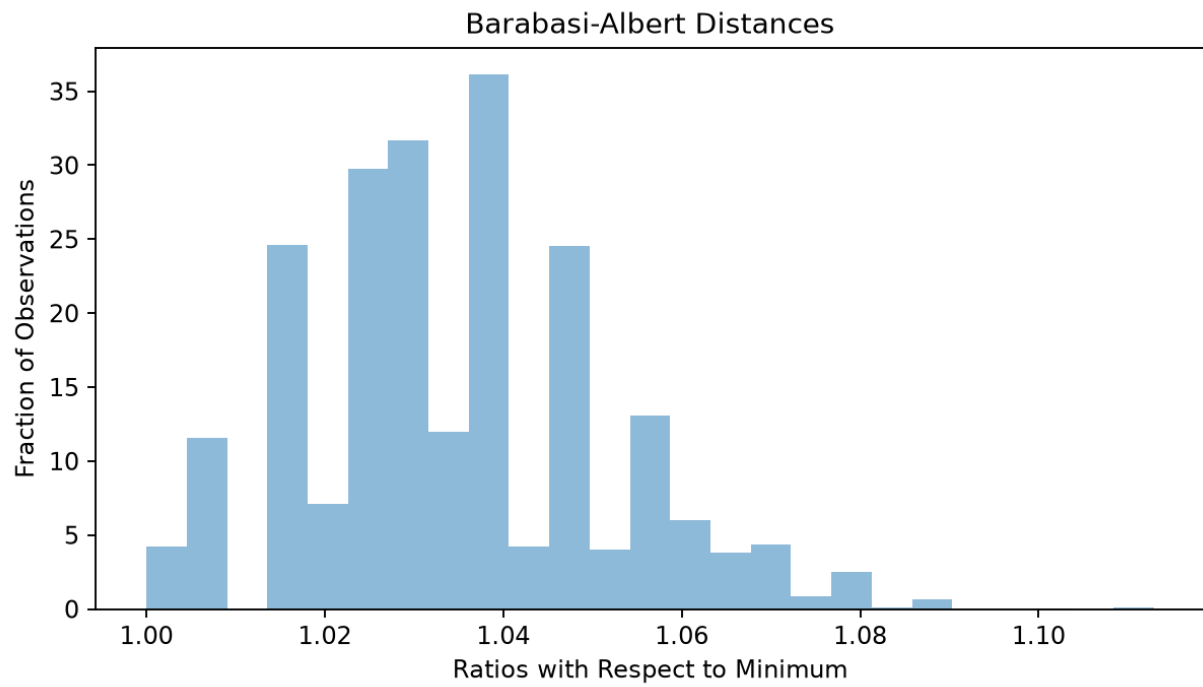
I ran the supplied `tspTests.py` unchanged, intercepting `plt.show()` to save its figures. NumPy and Python random seeds were both 65004. Each of three families used 50 independently generated distance matrices with 100 cities. Each matrix was tested at every starting city, yielding 5,000 ratios per family and 25 histogram bins. Each denominator is the minimum nearest-neighbor tour observed on that matrix, not a proven optimal TSP distance.

Family	Mean	Median	95th percentile	Maximum
Barabasi-Albert	1.034	1.032	1.065	1.113
Watts-Strogatz	1.092	1.085	1.198	1.317
Uniform Random	1.421	1.411	1.772	2.188

Barabasi-Albert ratios cluster tightly near 1: the average tour is about 3.45% longer than the best start for its matrix. Watts-Strogatz has a wider distribution, averaging about 9.20% above its best start. Uniform Random is much more sensitive to start: the mean is about 42.09% above the best observed start, with a maximum ratio of 2.188. The uniform matrices are asymmetric and need not satisfy the triangle inequality; an inexpensive greedy edge can leave an expensive closing edge.

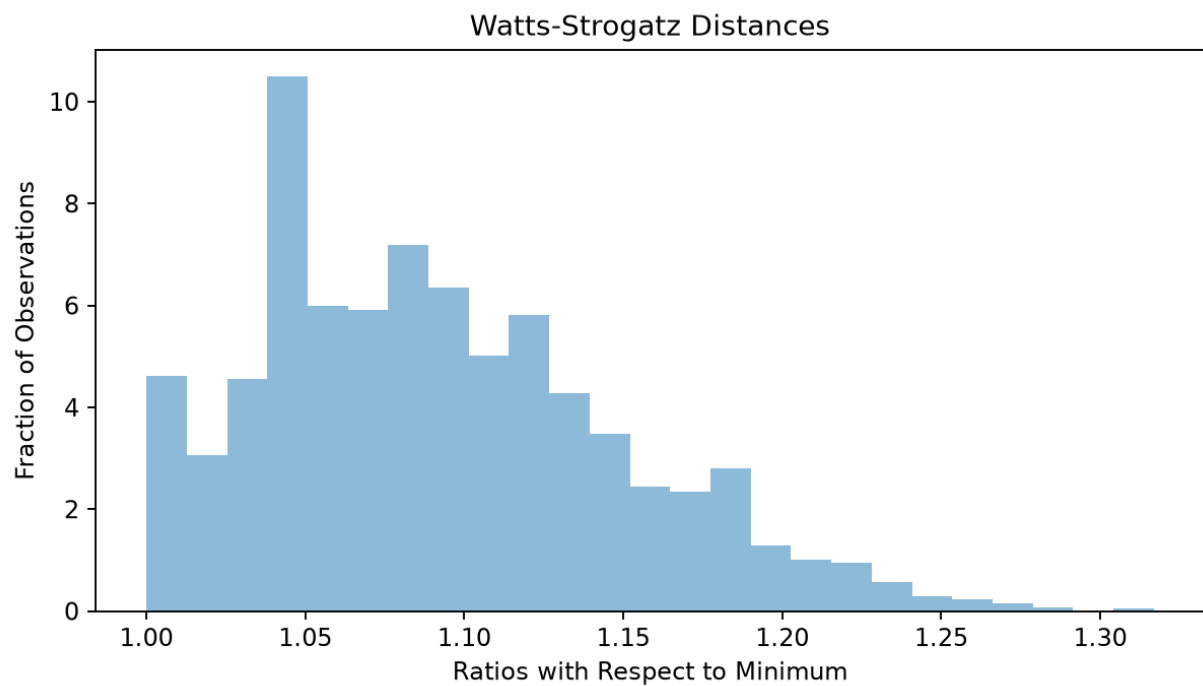
All starts on one matrix share its denominator and are dependent observations. These histograms describe these sampled instances; they do not establish approximation guarantees. Trying several starts can improve the observed nearest-neighbor result.

1(c). Barabasi-Albert histogram



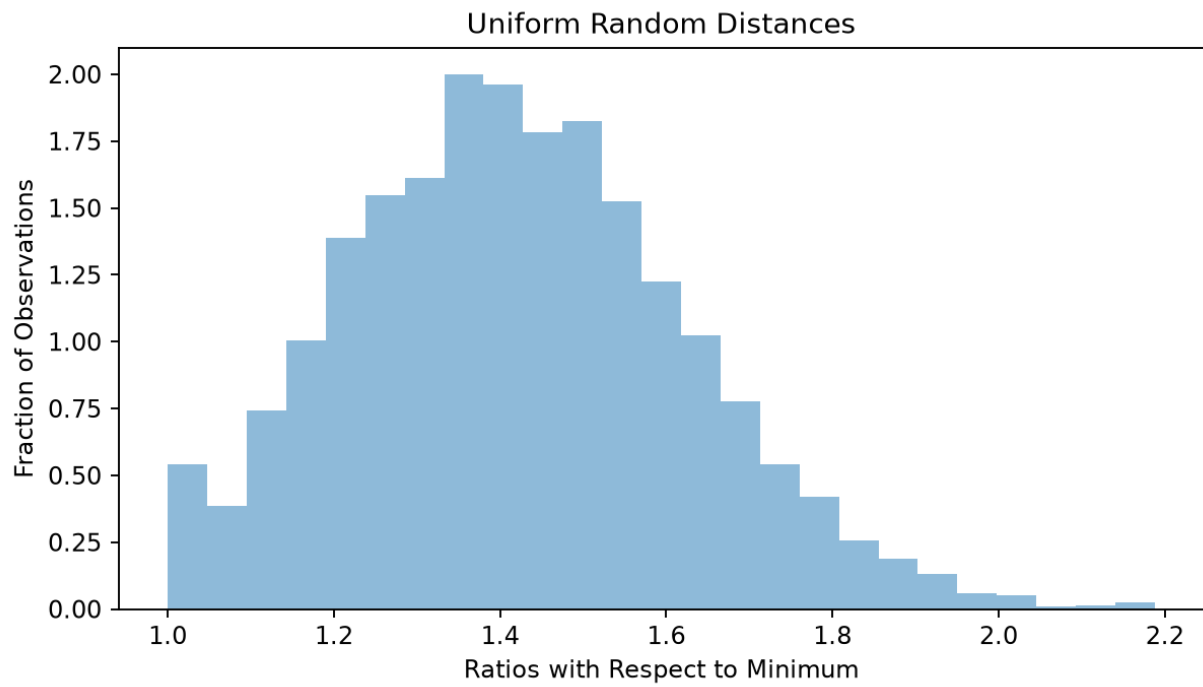
Horizontal axis: tour distance / minimum distance on the same matrix. Vertical axis: probability density, matching density=True in the supplied script.

1(c). Watts-Strogatz histogram



Horizontal axis: tour distance / minimum distance on the same matrix. Vertical axis: probability density, matching density=True in the supplied script.

1(c). Uniform Random histogram



Horizontal axis: tour distance / minimum distance on the same matrix. Vertical axis: probability density, matching density=True in the supplied script.

1(d). Candidate-generation functions

Each function copies its input and preserves the route's city multiset. For adjacent swaps I use consecutive list indices, without wrapping the last index to the first. The range functions choose distinct endpoints and include both endpoints. Paths with fewer than two cities return a copy unchanged.

```
def swapPairs(path, numSwaps=1, adj=False):
    """Return a copy with random swaps; adjacent means consecutive indices."""
    candidate = list(path)
    n = len(candidate)
    if n < 2:
        return candidate
    for _ in range(numSwaps):
        if adj:
            a = int(np.random.randint(n - 1))
            b = a + 1
        else:
            a, b = np.random.choice(n, size=2, replace=False)
            candidate[a], candidate[b] = candidate[b], candidate[a]
    return candidate

def permuteRange(path):
    """Shuffle an inclusive slice between two distinct random indices."""
    candidate = list(path)
    if len(candidate) < 2:
        return candidate
    a, b = sorted(np.random.choice(len(candidate), size=2, replace=False))
    candidate[a:b + 1] = np.random.permutation(candidate[a:b + 1]).tolist()
    return candidate

def reverseRange(path):
    """Reverse an inclusive slice between two distinct random indices."""
    candidate = list(path)
    if len(candidate) < 2:
        return candidate
    a, b = sorted(np.random.choice(len(candidate), size=2, replace=False))
    candidate[a:b + 1] = candidate[a:b + 1][::-1]
    return candidate
```

A random permutation may return the original slice; multiple swaps may undo one another. Such proposals remain valid. With a fixed number of swaps, all three functions take $O(n)$ time because they copy the route. NumPy choice without replacement may construct a permutation of the population, so a conservative bound for swapPairs is $O(n \cdot (1 + \text{numSwaps}))$ in general. Each uses $O(n)$ auxiliary space.

1(e). Simulated annealing

Initialize with a random permutation. Always accept a candidate of equal or lower distance. Accept an uphill change of size δ with probability $\exp(-\delta/T)$ when T is positive. Multiply T by α after each iteration. Keep the best-ever tour separately from the current tour; uphill moves may worsen the current tour. At $T=0$ only non-increasing moves are accepted. `distList` records the current distance after each iteration, and has exactly `maxIterations` entries.

```
def runSimulatedAnnealing(D, genCandidate=swapPairs,
                          maxIterations=1000, T=-1, alpha=0.9):
    """Return best tour, its distance, and current distance after each step."""
    if T == -1:
        T = float(np.sqrt(len(D)))
    T = float(T)
    if T < 0 or not np.isfinite(T):
        raise ValueError("T must be finite and nonnegative, or -1")
    if not 0 < alpha <= 1:
        raise ValueError("alpha must lie in (0, 1]")
    if maxIterations < 0:
        raise ValueError("maxIterations must be nonnegative")
    curPath = np.random.permutation(len(D)).tolist()
    curDist = totalDistance(D, curPath)
    bestPath, bestDist = curPath[:], curDist
    distList = []
    for _ in range(maxIterations):
        candidate = genCandidate(curPath[:])
        candidateDist = totalDistance(D, candidate)
        delta = candidateDist - curDist
        if delta <= 0 or (T > 0 and np.random.random() < np.exp(-delta / T)):
            curPath, curDist = list(candidate), candidateDist
            if curDist < bestDist:
                bestPath, bestDist = curPath[:], curDist
            distList.append(curDist)
        T *= alpha
    return bestPath, bestDist, distList
```

For the provided candidate functions, candidate construction, tour scoring, and any route copy each take $O(n)$ time. Thus i iterations take $O(n + i \cdot n)$, or $O((i+1) \cdot n)$, for n at least 1. Including degenerate empty inputs gives $O((i+1) \cdot (n+1))$. Auxiliary space is $O(n+i)$, including the returned distance history. With a general generator taking $G(n)$ time, the bound becomes $O(n + i \cdot (n+G(n)))$.

1(f). Annealing experiments and tuning

For each family the supplied script generates one 100-city matrix and runs each candidate function for 10,000 iterations on that same matrix. Each run has its own random initial tour, $T=10$, and $\alpha=0.9$. These are single runs per combination, so the comparison is descriptive and is confounded by different initial tours.

Family	Generator	Initial	Best / final
Barabasi-Albert	swapPairs	239.000	132.000
Barabasi-Albert	permuteRange	233.000	175.000
Barabasi-Albert	reverseRange	236.000	112.000
Watts-Strogatz	swapPairs	494.000	176.000
Watts-Strogatz	permuteRange	499.000	274.000
Watts-Strogatz	reverseRange	529.000	121.000
Uniform Random	swapPairs	52.781	10.147
Uniform Random	permuteRange	51.351	19.597
Uniform Random	reverseRange	49.458	23.863

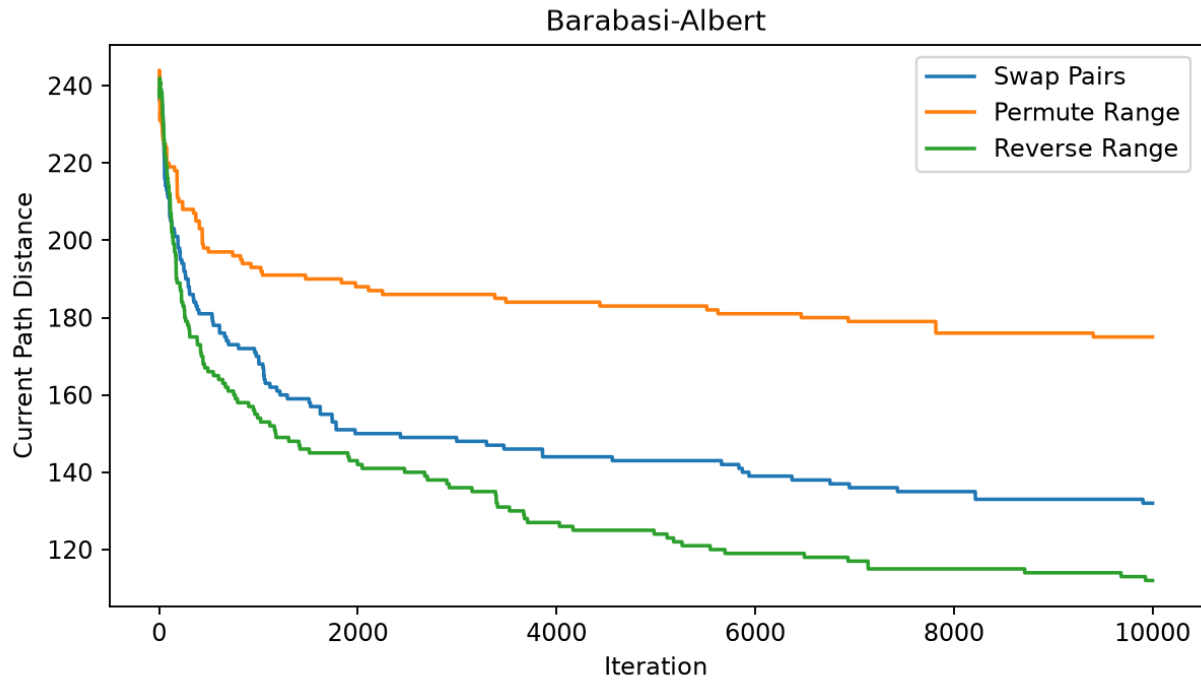
All runs improve substantially and end at their best recorded distances in this experiment. Reversal performs best on the two symmetric graph-distance instances (112 and 121), while pair swaps perform best on the asymmetric uniform instance (10.147). In a symmetric matrix, reversing a segment preserves its internal edge costs and changes just the boundary edges, resembling a 2-opt move. In an asymmetric matrix, reversal also reverses all its internal arcs and can be much more disruptive. Large random permutations often disturb many good edges.

The plotted current distance can rise during early uphill acceptances, but settles into a mostly decreasing staircase as cooling proceeds. The default cools rapidly: T after 100 updates is about 0.000266, and after 200 about 0.00000000706. Almost all the 10,000 iterations therefore behave like greedy local improvement. A late plateau indicates that accepted improvements are rare under this schedule; it does not certify optimality.

To tune, first sample positive candidate distance changes and choose T_0 near $-\delta_{\text{typical}} / \log(p_0)$, for example $p_0=0.8$. Choose a final temperature and set $\alpha=(T_{\text{final}}/T_0)^{(1/i)}$ so cooling spans the actual iteration budget. Compare multiple candidate operators, temperatures, schedules, and restart counts using equal evaluation budgets and repeated paired seeds on the same matrices and initial tours. Compare the distribution of best distances and runtime, then evaluate the chosen setting on fresh instances. Slower cooling may help escape local minima but is not guaranteed to improve every run.

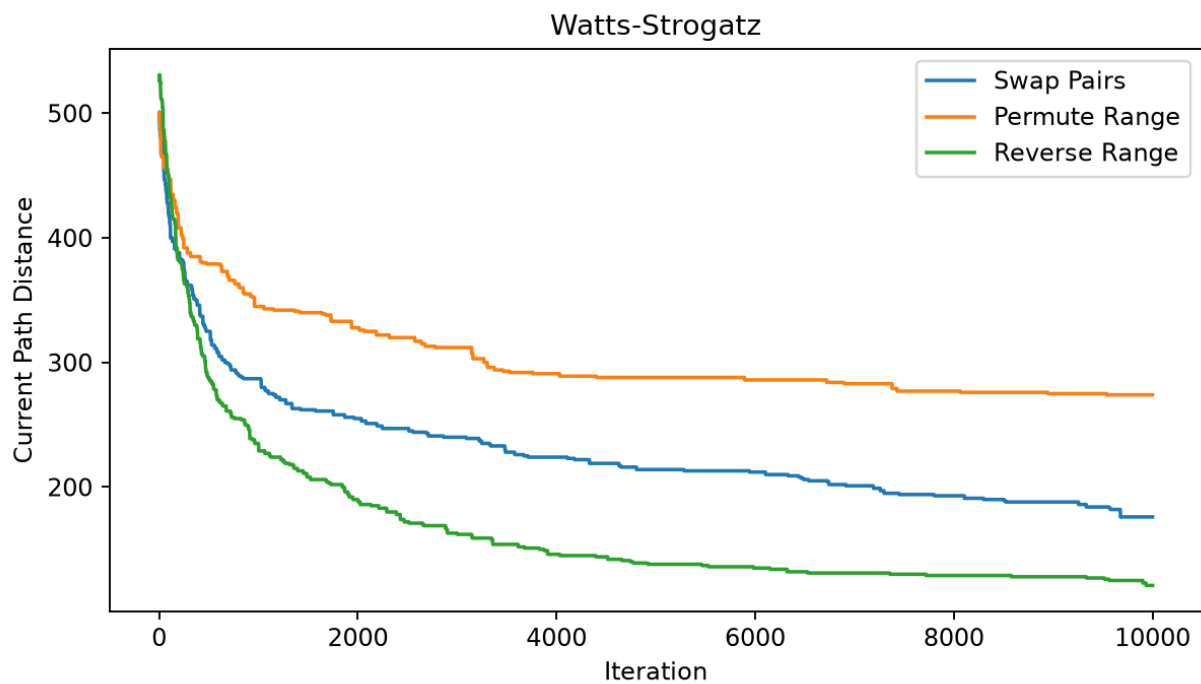
Script caveats: randomDistances ignores its low/high arguments and actually produces values in $[0,1]$; these plots use that actual implementation. The script's unit-square generator uses $\text{size}=[n,n]$ for $d=2$, making n points in n dimensions, and the square-region generator rounds down to a multiple of nine. Those two families were not selected in the supplied main experiment. The histogram label says fraction, but $\text{density}=\text{True}$ produces a probability density.

1(f). Barabasi-Albert annealing trajectory



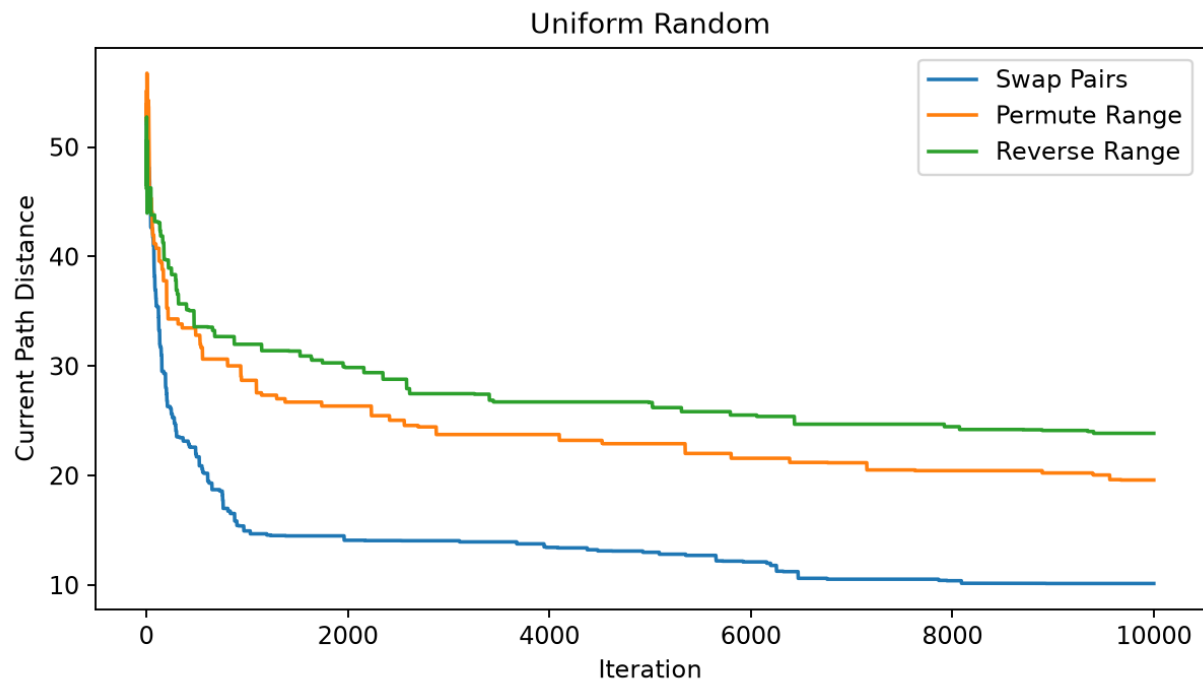
Horizontal axis: iteration. Vertical axis: current closed-tour distance. The PDF figure includes all 10,000 history entries; the standalone LaTeX drawing samples the trajectory for compactness.

1(f). Watts-Strogatz annealing trajectory



Horizontal axis: iteration. Vertical axis: current closed-tour distance. The PDF figure includes all 10,000 history entries; the standalone LaTeX drawing samples the trajectory for compactness.

1(f). Uniform Random annealing trajectory



Horizontal axis: iteration. Vertical axis: current closed-tour distance. The PDF figure includes all 10,000 history entries; the standalone LaTeX drawing samples the trajectory for compactness.

2(a). Why Busy Beaver is uncomputable

Use the course's step-count convention: $BB(n)$ is the largest number of steps taken before halting by any n -working-state, binary-tape Turing machine started on an all-zero tape; a separate halt state is not counted. This is consistent with the previous assignment's $BB(2)=6$ convention. It is not the alternative definition that maximizes the number of printed ones.

For each fixed n there are finitely many machines, and finitely many halting times among those that halt, so this maximum is finite. However, simulating all those machines longer and longer cannot in general distinguish a machine that runs forever from one that eventually halts after an enormous delay. A computable maximum halting-time bound would remove that difficulty and decide halting, which is impossible.

Consequently there is no algorithm guaranteed to compute $BB(n)$ for every n . Halting examples establish lower bounds, and longer searches may improve them. A finite cutoff cannot generally prove that all remaining machines fail to halt. An exact value requires ruling out every competing machine above the claimed maximum, typically through proofs of nontermination or bounded halting behavior. Some individual small values can be established; incomputability concerns one uniform algorithm for all n .

2(b). Reduction from the halting problem

Assume for contradiction that $BB(n)$ is computable and always halts. Given an arbitrary machine M and input w , effectively compile a binary-tape, blank-start machine N that first writes an encoding of w , then simulates M on w . The compiler uses finitely many additional working states and preserves halting: N halts on its blank tape if and only if M halts on w . Count the working states of this constructed machine; do not use the unmodified state count of M .

```
def decideHalting(M, w):
    # Effective finite construction, not a halting oracle:
    N = compileToBinaryBlankStart(M, w)
    n = numberOfWorkingStates(N) # halt state excluded
    limit = BB(n) # hypothetical total function
    config = blankConfiguration(N)
    if isHalted(config):
        return True
    for _ in range(limit):
        config = step(N, config)
        if isHalted(config):
            return True
    return False
```

The initial halt check handles zero steps, and checking after each transition includes halting at exactly limit steps. The hypothetical BB call terminates, and the loop is bounded, so `decideHalting` always terminates. If N halts, its halting time cannot exceed $BB(n)$, and the simulation accepts. If it has not halted within that bound, it cannot halt later without contradicting the definition of $BB(n)$, and the function rejects. Therefore this procedure would decide the general halting problem. Since that problem is undecidable, a total computable BB function cannot exist.

Even restricting to blank-input machines is already undecidable: the input-writing wrapper reduces arbitrary-input halting to blank-input halting. If a different source uses the printed-ones Busy Beaver definition, this particular time-bound loop needs an additional reduction; one must not treat a bound on printed ones as a bound on steps.

Validation and reproducibility

The implementation passed 434 correctness/property checks. These cover directed return edges, deterministic nearest-neighbor ties, empty/single-city inputs, permutation preservation, input immutability, adjacent swaps, zero iterations, greedy cooling, uphill acceptance, and retaining the best-ever route. The supplied `tspTests.py` generates plots rather than asserting correctness.

Dependencies used: NumPy 2.5.3, NetworkX 3.7, Matplotlib 3.11.2. `experiment_results.json` records the seed, summary statistics, and complete annealing histories. Reproduction scripts and dependency versions accompany the implementation. No submission to Canvas or INGINIOUS was performed.